

Expt 11 IIR Filters using DSP Processor

11.1 Design and Implementation of an IIR Low-Pass Filter using DSP Processor

Objectives

1. To design an IIR Low-Pass Filter with the given filter specifications.
2. To implement the designed IIR Low-Pass Filter on the DSP processor (TMS320C5416 Starter Kit).
3. To apply an input signal to the filter and obtain the filtered output using the DSP processor.
4. To analyze and verify the performance of the IIR Low-Pass Filter by comparing the input and output responses.

Hardware and Software Required:

DSP – TMS320VC5416-5416 Board

Cpu

ME Universal Debugger DSP – 5416

Procedure:

1. Connect the serial port cable between the PC and the VSK-5416 kit.
2. Switch ON the VSK-5416 kit and press the RESET button.
3. Open Vi Universal Debugger.
4. Select Serial → Port Settings → Auto Detect and verify that the message "Vi Universal Debugger found in COMx" appears.
5. Create a new project (File → New Project) and assign a project name.
6. Create a new source file (File → New File) and write the program.
7. Save the source file.
8. Add the source file to the project.
9. Add the command file (Micro5416.cmd) to the project.
10. Save the project.
11. Build/Rebuild the project and ensure there are no errors.
12. Download the executable file to the VSK-5416 kit.
13. Run the program.
14. Observe and verify the output.

LOW PASS FILTER (IIR):

;=====

; 2nd Order Butterworth IIR Low-Pass Filter

; Sampling Frequency = 62 kHz

; Cutoff Frequency = 4 kHz

;=====

; Variable declaration for the filter

XN .SET 10H

XNM1 .SET 11H

XNM2 .SET 12H

YN .SET 13H

YNM1 .SET 14H

YNM2 .SET 15H

SUM .SET 16H

; Filter coefficients

A1 .SET 0E903H

A2 .SET 0905H

B0 .SET 082H

B1 .SET 0104H

B2 .SET 082H

.mmregs

.text

START:

STM #0140H, ST0 ; Initialize data page

RSBX CPL ; Reset compiler mode bit

RSBX FRCT ; Reset fractional mode bit

NOP

NOP

NOP

;-----

; Initialize x(n), x(n-1), x(n-2)

; y(n), y(n-1), y(n-2)

;-----

ST #0H, XN

ST #0H, XNM1

ST #0H, XNM2

ST #0H, YN

ST #0H, YNM1

ST #0H, YNM2

REPEAT:

;-----

; ADC Sampling

;-----

PORTR 6,0 ; Start ADC conversion

NOP

NOP

NOP

NOP

PORTR 4,0 ; Read ADC sampled data

```

LD    0,A          ; Load ADC data into accumulator
AND   #0FFFH,A     ; Extract 12-bit ADC data
XOR   #0800H,A     ; Correct two's-complement format
SUB   #0800H,A     ; Convert to processor format

```

```

STL   A,XN         ; Store current input x(n)

```

```

;-----
; IIR Filter Equation
;
;  $y(n) = b_0 \cdot x(n) + b_1 \cdot x(n-1) + b_2 \cdot x(n-2)$ 
;   -  $a_1 \cdot y(n-1) - a_2 \cdot y(n-2)$ 
;-----

```

```

LD    #0H,B        ; Clear B accumulator

```

```

;  $b_0 \cdot x(n)$ 

```

```

LD    XN,A
STLM  A,T
MPY   #B0,A
LD    A,B

```

```

;  $b_1 \cdot x(n-1)$ 

```

```

LD    XNM1,A
STLM  A,T
MPY   #B1,A
ADD   A,B

```

```

;  $b_2 \cdot x(n-2)$ 

```

```

LD    XNM2,A

```

```
STLM  A,T
MPY   #B2,A
ADD   A,B
```

```
; -a1*y(n-1)
LD    YNM1,A
STLM  A,T
MPY   #A1,A
SUB   A,B
```

```
; -a2*y(n-2)
LD    YNM2,A
STLM  A,T
MPY   #A2,A
SUB   A,B
```

```
;-----
; Scale result
;-----
```

```
SFTL  B,-12      ; Shift result right by 12 bits
STL   B,YN       ; Store filtered output y(n)
```

```
;-----
; Update previous samples
;-----
```

```
LD    YNM1,A
STL   A,YNM2     ; y(n-2) = y(n-1)
```

```
LD    YN,A
```

STL A,YNM1 ; $y(n-1) = y(n)$

LD XNM1,A

STL A,XNM2 ; $x(n-2) = x(n-1)$

LD XN,A

STL A,XNM1 ; $x(n-1) = x(n)$

;

; Send filtered output to DAC

;

LD YN,A

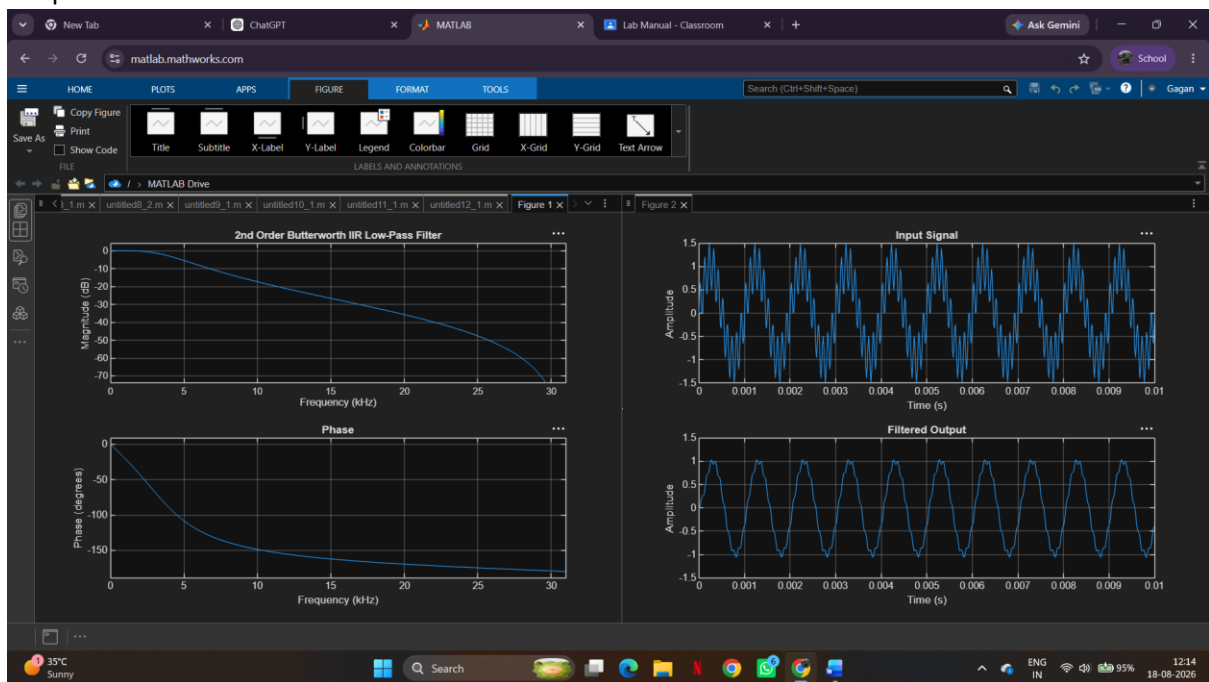
ADD #800H,A ; Add DAC offset

STL A,YN

PORTW YN,04H ; Send output to DAC

B REPEAT ; Repeat continuously

Output:



Result:

1. The IIR Low-Pass Filter was successfully designed according to the given specifications.
2. The designed filter was implemented on the TMS320C5416 DSP Starter Kit.
3. The input signal was processed through the IIR Low-Pass Filter, and the filtered output was obtained.
4. The output confirmed that low-frequency components were passed while high-frequency components were attenuated, verifying the correct operation of the filter.